

Florence: Architecture - Platform Service

Overview

The Florence Platform Service is a highly scalable cross-platform frontend application built with the Flutter framework. It provides the primary user interface for all system participants including patients, clinicians and administrators. The application delegates complex processing to external backend services to manage health data, trigger artificial intelligence pipelines and display real-time clinical insights. The system enforces a strict separation of concerns where the frontend handles state representation and the backend manages data integrity.

Architecture and State Management

The application follows a feature-first architectural pattern to group related screens, providers and data models. It heavily utilises the Riverpod package for reactive state management, dependency injection and asynchronous data caching. State is primarily managed via `AsyncNotifier` and `FutureProvider` implementations to gracefully handle loading and error states during network operations. The application automatically invalidates these cached data providers upon sign-in or sign-out to guarantee state isolation between different user sessions.

The system connects directly to the Supabase client solely for secure user authentication, deep link resolution and session persistence. All clinical data operations route through a custom singleton `ApiService` that intercepts HTTP requests to append the active JSON Web Token to the authorisation header. This service includes internal logic to catch HTTP 401 responses, trigger a token refresh via the Supabase client and automatically retry the failed request.

Routing and Deep Linking

The application implements a central routing generator to handle navigation requests. The router evaluates the requested path and yields specific transition animations based on the context. Patient and clinician interfaces utilise native mobile page routes for fluid navigation while the administrative portal utilises zero-duration page builders to simulate a responsive

web environment. The system integrates the `app_links` package to intercept incoming universal links. This allows the application to capture refresh tokens from email verification links and manually establish secure sessions.

User Interface and Responsive Design

The platform implements a robust layout system via `LayoutBuilder` and custom responsive helpers to adapt seamlessly across mobile, tablet and desktop environments. The system dynamically adjusts grid columns, font sizes and padding metrics based on the active viewport. It utilises distinct Material 3 theme configurations for different user roles to provide immediate visual context.

The application relies on the `fl_chart` package to render interactive Cartesian grids including line charts, bar charts and scatter plots for health analytics. For highly specialised visualisations, the application implements low-level `CustomPainter` classes. These painters calculate trigonometric angles to render the body mass index gauge, the cholesterol ratio pie chart and the particle burst animation triggered by the vitality index.

Core Configuration

The system requires specific environment variables to resolve backend endpoints and authenticate with the infrastructure. The developer must supply these values during the compilation process using Dart define flags.

- `DATA_SERVICE_URL` : The endpoint for the primary data backend.
 - `LLM_CHATBOT_SERVICE_URL` : The endpoint for the conversational agent.
 - `LLM_ENGINE_SERVICE_URL` : The endpoint for the artificial intelligence engine.
 - `SUPABASE_URL` : The project URL provided by the authentication host.
 - `SUPABASE_ANON_KEY` : The public key required for client-side authentication.
-

Directory Structure

The repository contains modular directories that segregate the frontend functionality by domain.

- `lib/main.dart` : The primary entry point that initialises the environment and application binding.
- `lib/app.dart` : The core widget that manages global routing and listens to authentication state transitions.
- `lib/config/` : A directory defining global routing maps, application constants and colour palettes.
- `lib/core/` : A directory housing global utilities, data formatters, input validators, network services and the responsive layout engine.
- `lib/shared/` : A directory containing reusable components such as custom buttons, loading indicators and empty state illustrations.
- `lib/features/patient/` : A directory containing the patient application including the unified dashboard, logging forms, analytics screens, medication trackers and chatbot interface.
- `lib/features/clinician/` : A directory containing the healthcare professional portal including caseload overviews, alert feeds and patient detail views.
- `lib/features/admin/` : A directory containing the administrative console including user management, organisation registration and the synthetic data simulator.

Security and Privacy

The platform enforces a strict zero-trust security model by delegating authentication, authorisation and data encryption in transit to the infrastructure layer.

- **Authentication and Session Management:** The application utilises Supabase Auth for secure user authentication, deep link resolution and session persistence. Password hashing and credential storage are handled entirely by the Supabase infrastructure.
- **Data Protection:** All clinical data operations route through a custom singleton `ApiService` that intercepts HTTP requests to append the active JSON Web Token (JWT) to the authorisation header. Data privacy and row-level access control are strictly enforced at the database layer via Supabase Row Level Security (RLS) policies, ensuring users can only access their own health records.
- **Administrative Access Control:** The administrative portal enforces strict Role-Based Access Control (RBAC) directly within the routing logic and widget trees. The system maps Supabase authentication metadata to internal `AdminRole` enumerators (e.g. Global Administrator, Hospital Administrator). The frontend utilises a custom `PermissionGuard` widget that intercepts rendering pipelines, hiding sensitive UI elements and blocking unauthorised navigation if the active user lacks specific `AdminPermission` claims.

Package Management

This repository contains a Dart and Flutter application. It relies on the standard Flutter pub package manager to resolve dependencies such as Riverpod, graphical charting libraries, Markdown renderers and image processing tools. The application integrates Shorebird to enable seamless over-the-air updates for deployed mobile applications.

Installation Instructions

The developer must follow these steps to prepare the local environment.

1. Install the Flutter software development kit on the host system.
2. Execute the package retrieval command to install the required dependencies.
3. Define the required environment variables during the build process or via a configuration file.
4. Launch the application on a web browser, simulator or physical device.

Core Services and Repositories

API and Network Client

The `ApiService` class operates as a singleton wrapper around the standard HTTP client. It automatically evaluates the active Supabase authentication session to inject Bearer tokens into outgoing request headers. The service maps specific uniform resource identifiers to backend targets, processes multipart file requests for image uploads and intercepts HTTP 401 responses to trigger seamless token refreshes.

Pattern Detection Engine

The `PatternDetectionService` evaluates the `HealthDataState` payload locally to identify clinical anomalies immediately after data entry. It is actively utilised by the local automation manager (`NotificationNotifier`) to trigger real-time UI alerts and persist clinical audit trails to the backend. The engine applies deterministic rules to evaluate the latest glucose readings against dynamic user-defined thresholds and detect prolonged physical inactivity (no activity

logged in 48 hours). It outputs `DetectedPattern` objects mapped to severity scales to drive the notification system.

Notification and Automation Handlers

The `NotificationNotifier` class persists a local array of `HealthNotification` objects. It listens to state transitions in the health data providers to dispatch alerts automatically. If the pattern detection engine identifies a critical event such as a hypertensive crisis or severe hypoglycaemia, the service instantly triggers a high-priority alert prompting the user to seek medical attention. The system transmits automated action records to the backend to maintain a verifiable clinical audit trail.

State Management Providers

The application relies on Riverpod providers to orchestrate data flow between the backend services and the user interface.

- `monitorDataProvider` : An asynchronous notifier that executes parallel HTTP requests to compile glucose readings, blood pressure logs, dietary records, physical activities and active diseases into a unified `HealthDataState` object.
- `recommendationProvider` : An asynchronous notifier that transmits a simplified health summary to the `LlmRecommendationService` . It parses the language model response into actionable `HealthRecommendation` models.
- `patientSettingsProvider` : A standard notifier that synchronises unit preferences such as millimoles per litre versus milligrams per decilitre. It triggers global state invalidation when units are modified to force chart recalculations.
- `chatProvider` : A notifier managing the chronological `ChatMessage` history. It streams user queries to the external chatbot service and appends the generated responses to the local array.

Routing Architecture

The application implements a central routing generator parsing URI string paths. It separates routing trees into general application routes and protected administrative routes.

Patient Routes

- `/dashboard` : The primary patient hub hosting the vitality index and intelligent insight widgets.
- `/log/glucose` , `/log/blood-pressure` , `/log/meal` , `/log/activity` : Dedicated form controllers with strict validators and artificial intelligence auto-fill triggers.
- `/trends` , `/hba1c-detail` , `/cholesterol-detail` : Complex data visualisation screens implementing `fl_chart` Cartesian grids to display historical progression against dynamic `PatientThreshold` boundaries.
- `/chat` : The conversational interface parsing Markdown strings from the language model service.
- `/recommendations` : The actionable health guidance feed displaying rationale explanations and triggering data references.

Clinician Routes

- `/clinician-dashboard` : The medical portal compiling priority alerts and an active patient roster.
- `/clinician/patient-detail` : The comprehensive oversight view allowing clinicians to append notes, override risk stratifications and manage patient medication cabinets.

Administrator Routes

- `/admin-dashboard` : The macro-level control centre featuring aggregate system metrics and audit logs.
- `/admin/patients` : The administrative table for manual clinician assignment and destructive data wiping operations.
- `/admin/organizations` : The registration interface for provisioning new hospital identifiers.
- `/admin/data-simulator` : The testing orchestrator that transmits simulation constraints to the language model engine to generate synthetic patient histories.

Interface and Rendering Engines

Responsive Layout System

The platform implements a distinct `ResponsiveLayoutSystem` class defining strict breakpoints at 600px, 1024px and 1440px. The system calculates cross-axis counts dynamically for grid views and switches between bottom navigation bars and persistent side rails depending on the available viewport width.

Custom Data Visualisation

The application bypasses standard widgets to render complex clinical visualisations using low-level canvas instructions.

- **Vitality Index Gauge:** The `_ScoreRingPainter` calculates trigonometric angles to draw a sweep gradient arc representing the patient health score.
- **Cholesterol Ratio:** The `_RatioPainter` renders segmented arcs to visually compare high-density lipoproteins against remaining cholesterol volume.
- **Body Mass Index Indicator:** The system plots dynamic bounds across a linear track, calculating fractional offsets to place the patient marker accurately within underweight, normal, overweight or obese zones.
- **Scatter Matrices:** The blood pressure analytics screen utilises `ScatterChart` implementations to plot systolic values against diastolic values, mapping standard deviation clusters against clinical danger zones.
- **Traffic Light Calendar:** The dietary impact view renders a twenty-eight day grid, calculating post-prandial glucose spikes to colour individual date cells green, yellow or red.